

Automated Rating Review System

1. Project Overview

An automated review system analyzes customer reviews and predicts the corresponding star ratings. Natural language processing (NLP) is used to extract sentiment and key features from the review texts.

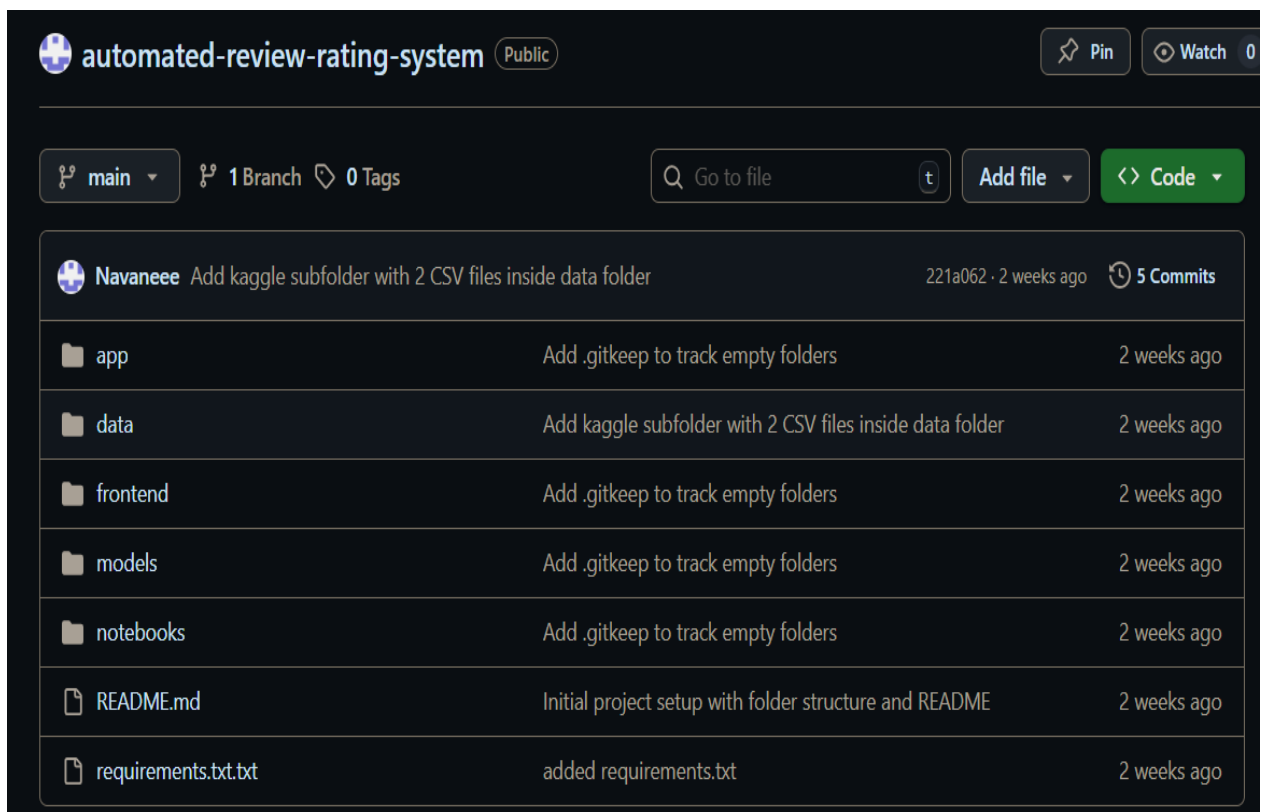
2. Environment Setup

- Installed Python 3.13.3 and the required libraries such as pandas, numpy, matplotlib, seaborn, and scikit-learn. For data preprocessing, NLP, and visualization, these packages were utilized.
- VS Code is used as the IDE.

3. GitHub Project Setup

Created a GitHub repository named automated-review-rating-system.

Structure of directory.



The screenshot shows the GitHub repository page for 'automated-review-rating-system'. The repository is public and has 1 branch and 0 tags. The commit history shows a series of commits by user Navaneee, with the most recent commit being 'Add kaggle subfolder with 2 CSV files inside data folder' 2 weeks ago. The file structure is listed below the commit history, showing folders for app, data, frontend, models, and notebooks, each with a commit message and date. The README.md file is also listed, with a commit message 'Initial project setup with folder structure and README' and a date of 2 weeks ago. The requirements.txt file is also listed, with a commit message 'added requirements.txt' and a date of 2 weeks ago.

File/Folder	Commit Message	Date
app	Add .gitkeep to track empty folders	2 weeks ago
data	Add kaggle subfolder with 2 CSV files inside data folder	2 weeks ago
frontend	Add .gitkeep to track empty folders	2 weeks ago
models	Add .gitkeep to track empty folders	2 weeks ago
notebooks	Add .gitkeep to track empty folders	2 weeks ago
README.md	Initial project setup with folder structure and README	2 weeks ago
requirements.txt	added requirements.txt	2 weeks ago

4. Data Collection

The dataset was collected from Kaggle and it consists of 568454 Amazon product ids, customer reviews, star ratings and 7 more features also.

Dataset link : https://github.com/Navaneeee/automated-review-rating-system/tree/main/data/kaggle/review_data.csv

5. Data Preprocessing.

5.1 Data Preview and Dropping Unnecessary Columns

The Amazon review data consisted of 10 different columns. It preprocessed to include only review ratings and review text. All other columns were removed. The preview is given below. (568454, 2) is the dimension of the new data.

```
data.head()
```

	rating	review_text
0	5	I have bought several of the Vitality canned d...
1	1	Product arrived labeled as Jumbo Salted Peanut...
2	4	This is a confection that has been around a fe...
3	2	If you are looking for the secret ingredient i...
4	5	Great taffy at a great price. There was a wid...

5.2 Missing Values

There were no missing values present in the data.

5.3 Duplicate Values Handling

There are 174779 number of duplicate rows were present in the data and all of them were removed with the code

```
data.drop_duplicates(inplace=True)
Also 96 duplicate reviews were present in review text column.Those were also
removed with the code.
data.drop_duplicates(subset=['review_text'], inplace=True)
```

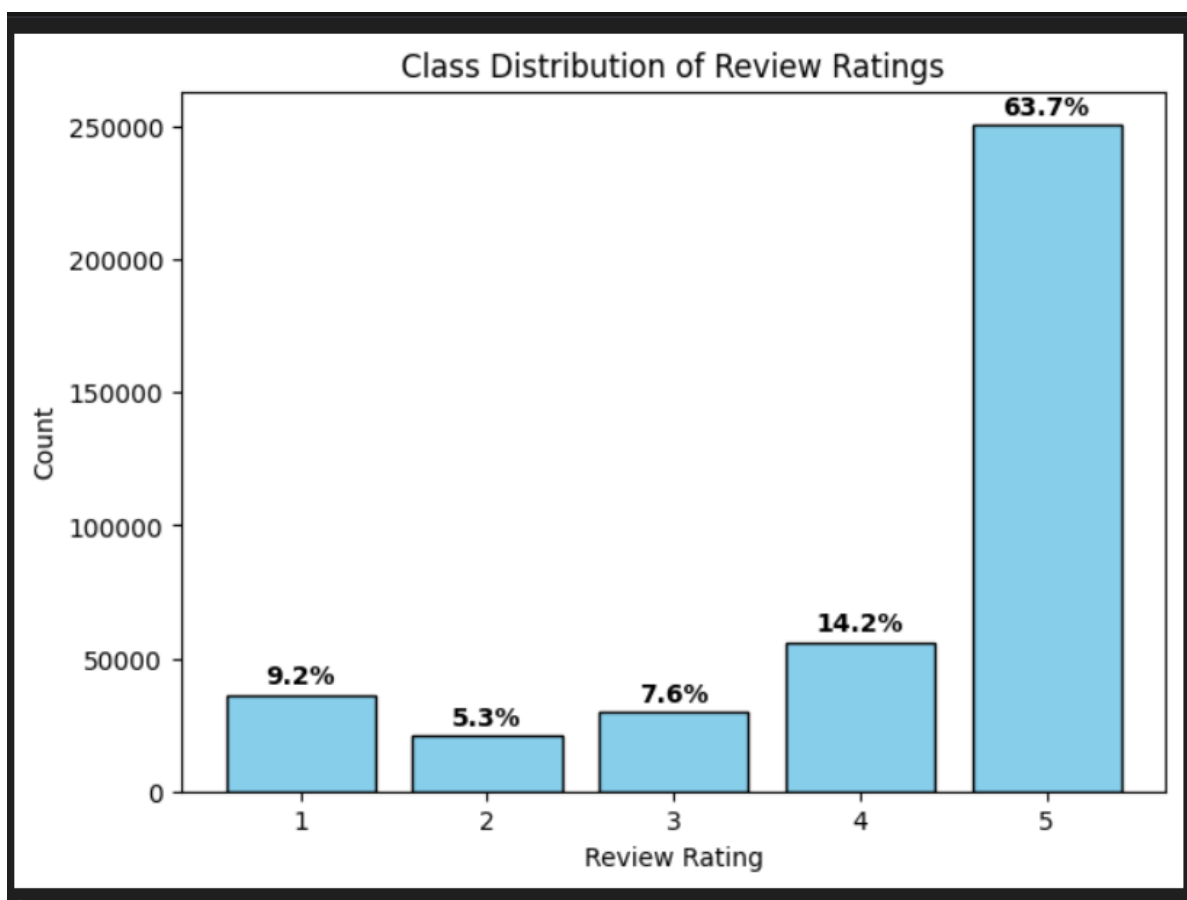
The final shape of the data was (393579, 2)

5.4 Class Distribution

The total value count of each rating is found with using `value_count()` method and it shown as

```
Class Distribution (Counts):  
rating  
1      36275  
2      20792  
3      29754  
4      56042  
5     250716  
Name: count, dtype: int64
```

The distribution of ratings is then plotted with a barplot.



The inferences from the barplot were;

1. The dataset is heavily imbalanced.
2. 5-star reviews dominate and it's around 64%
3. 2-star reviews are very low, around 5%
4. To make the data balance, we try random downsampling to the high ratings, 250,716 ratings are reduced to 50,000 and all other ratings are increased to 50,000.

5.5 Splitting the dataset to Balanced data and Imbalanced data.

The whole data is split into a balanced data with the code

```
#define the sample size(half of class 2 star ratings' count)
sample_size=class_counts[2]//2

#Sample from rating 2
#Selecting all the rows where rating column =1 and sample size is half of lowest
count class
df2_list=data[data['rating']==2].sample(n=sample_size,random_state=42) #ensures
the sampling is reproducible

#Sample from rating 1,3,4,5
#selecting same number of sample size rows from all other ratings' class
dfr_list=[data[data['rating']==r].sample(n=sample_size,random_state=42) for r in
[1,3,4,5]]

#concatenate df2_list with all others( df2,[df1,df3,df4,df5])
balanced_dataset = pd.concat([df2_list] + dfr_list)

#Create second dataset consisting of remaining rows
second_dataset = data.drop(balanced_dataset.index)

#Shuffle the combined dataset
balanced_dataset = balanced_dataset.sample(frac=1,
random_state=42).reset_index(drop=True)

#Saving the balanced dataset.
balanced_dataset.to_csv(r"C:\Users\kjinav\Documents\Navaneet\automated-review-
rating-system\data\cleaned\balanced_data.csv", index=False)
```

And split into an imbalanced dataset with code;

```
#setting the target percentages
percentage_dict={1:0.10,      # 1 *      =10%
                2:0.15,      # 2 **     =15%
                3:0.25,      # 3 ***   =25%
                4:0.30,      # 4 ****  =30%
                5:0.20}      # 5 ***** =20%

#total number of rows in the dataset
total=len(second_dataset)

#create an empty list to keep new data
new=[]

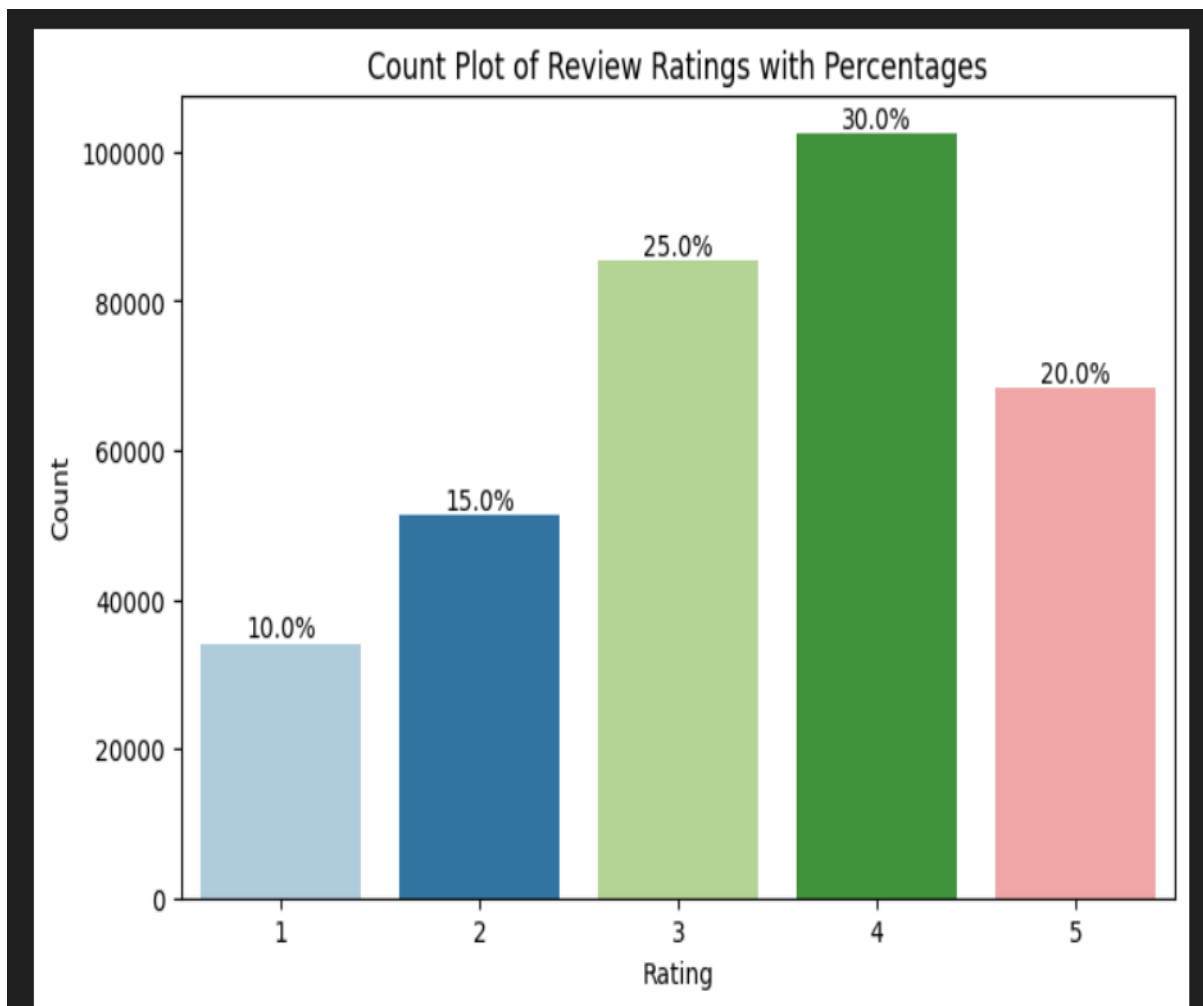
#for each ratings,we take number of rows we want according to the target
percentage
for ratings,percentage in percentage_dict.items():
    total_rows_needed=int(round(percent*total))
```

```
rows=second_dataset[second_dataset['rating']==ratings]
#select only that rating(filtering the rows)
sample_rows=rows.sample(total_rows_needed,replace=True,random_state=42)
new.append(sample_rows)
```

Thereafter saved,naming imbalanced_data

```
#save this imbalanced dataset.
imbalanced_data.to_csv(r"C:\Users\kjinav\Documents\Navaneee\automated-review
rating-system\data\cleaned\imbalanced_data.csv", index=False)
```

The countplot of imbalanced data



6. Imbalanced Data

Dataset link : https://github.com/Navaneee/automated-review-rating-system/tree/main/data/kaggle/imbalanced_data.csv

6.1 Data Import and Preprocessing

This stage involves importing the imbalanced dataset, inspecting its structure, handling missing values, and performing initial cleaning steps such as removing duplicates and computing word counts. The preview of the data just given.

	rating	review_text
0	5	Love this product, however the Amazon price is...
1	4	This has recently become my staple breakfast c...
2	4	This spread is just right but costs only about...
3	2	I guess I learned from this purchase that rais...
4	2	This coffee is sort of grocery store quality-a...

6.2 Importing All the Necessary Libraries

The following Python libraries were used for text preprocessing, feature extraction, and dataset splitting.

Library	Purpose
string	Contains string constants and utility functions. Used to remove punctuation from text.
nltk	Natural Language Toolkit for NLP tasks such as tokenization, stemming, and lemmatization.
from nltk.corpus import stopwords	Provides a list of common stopwords (e.g., "the", "is", "and"), which are removed to reduce noise.
from nltk.stem import WordNetLemmatizer	Reduces words to their base or dictionary form (lemmatization).
from sklearn.model_selection import train_test_split	Splits datasets into training and testing sets for model evaluation.
from sklearn.feature_extraction.text import TfidfVectorizer	Converts textual data into numerical features using TF-IDF, representing the importance of words across documents.

6.3 Text Preprocessing

Created a user defined function called “preprocess_text”. This function cleans and prepares the review text for analysis using Natural Language Processing (NLP) techniques.

1. Lowercasing:

Converts all text to lowercase to maintain consistency.

2. Removing URLs and HTML tags:

Eliminates links and HTML elements that are not useful for text analysis.

3. Removing emojis and special characters:

Keeps only alphabetic characters to focus on meaningful words.

4. Tokenization:

Splits the text into individual words (tokens).

5. Stopword Removal:

Commonly used words that do not contribute much to the meaning or sentiment of a sentence are removed. The list of stopwords used comes from **NLTK's** default English stopwords. Removing these words helps reduce noise and improves the focus on meaningful terms for tasks like text classification, sentiment analysis, or information retrieval. **NLTK** provides a simple and lightweight stopword list ready to use, making it ideal for quick preprocessing in research or small projects.

spaCy also offers stopword lists, but it is more resource-intensive and geared toward full-featured NLP pipelines. For tasks where only stopword removal is required, NLTK is faster and easier to implement. The list of stopwords used comes from NLTK's default English stopwords set, which includes words such as:

```
{'by', 'yourself', 've', 'been', 'other', 'in', 'didn't', 'each', 'this', 'he'll', 'am', 'they've', 'you're', 'ourselves', 'ours', 'doing', 'it'll', 'shan't', 'hadn', 'most', 'than', 'they', 'we'd', 'she', 'haven', 'couldn't', 'hasn't', 'his', 'out', 'over', 'we'll', 'isn't', 'against', 'which', 'hers', 'or', 'won't', 'you'd', 'above', 'should', 'on', 'because', 'he', 'she'll', 'does', 'couldn', 'shan', 'don't', 'they'd', 'with', 'a', 'why', 'weren', 'shouldn', 'yourselves', 'ain', 'she'd', 'more', 'll', 'myself', 'theirs', 'but', 'about', 'them', 'did', 'to', 'has', 'be', 'so', 'of', 'too', 'while', 'should've', 'those', 'don', 'she's', 'some', 'all', 'such', 'aren't', 'from', 'here', 'whom', 'mightn't', 'wouldn', 'm', 'you've', 'at', 'themselves', 'mustn', 'if', 'then', 'now', 'nor', 'that', 'they're', 'won', 'no', 'have', 'me', 'were', 'own', 'we've', 'any', 'o', 'do', 'mightn', 'we're', 'he's', 'shouldn't', 'further', 'was', 'there', 'is', 'its', 'between', 'for', 'only', 'an', 'my', 'i', 'haven't', 'where', 'hadn't', 't', 'what', 'into', 'once', 'below', 'himself', 'having', 'are', 'ma', 'off',
```

'we', 'who', 'before', 'yours', 'hasn', 'not', 's', 'as', 'doesn', 'didn', 'him', 'that'll', 're', 'it'd', 'isn', 'it', 'can', 'her', 'd', 'weren't', 'had', 'itself', 'their', 'herself', 'you'll', 'again', 'you', 'down', 'your', 'i'd', 'it's', 'through', 'under', 'very', 'wasn', 'during', 'how', 'wasn't', 'mustn't', 'i've', 'both', 'until', 'y', 'i'm', 'and', 'the', 'after', 'when', 'few', 'aren', 'our', 'they'll', 'up', 'being', 'i'll', 'needn', 'these', 'same', 'will', 'wouldn't', 'he'd', 'just', 'doesn't', 'needn't'}

6. Lemmatization

Lemmatization is the process of converting each word to its base or dictionary form (known as the lemma). For example:

“walking” → “walk”,

“better” → “good”.

This step helps ensure that words with similar meanings are treated as the same during analysis, thereby improving model consistency and reducing feature sparsity. While both stemming and lemmatization aim to reduce words to their root form, lemmatization is more accurate and linguistically meaningful.

7. Rejoining:

The cleaned tokens are combined back into a single string. The cleaned text is saved in a new column “cleaned_review”, and a “word_count” column is created to store the number of words in each processed review.

8. Removed too long or short texts:

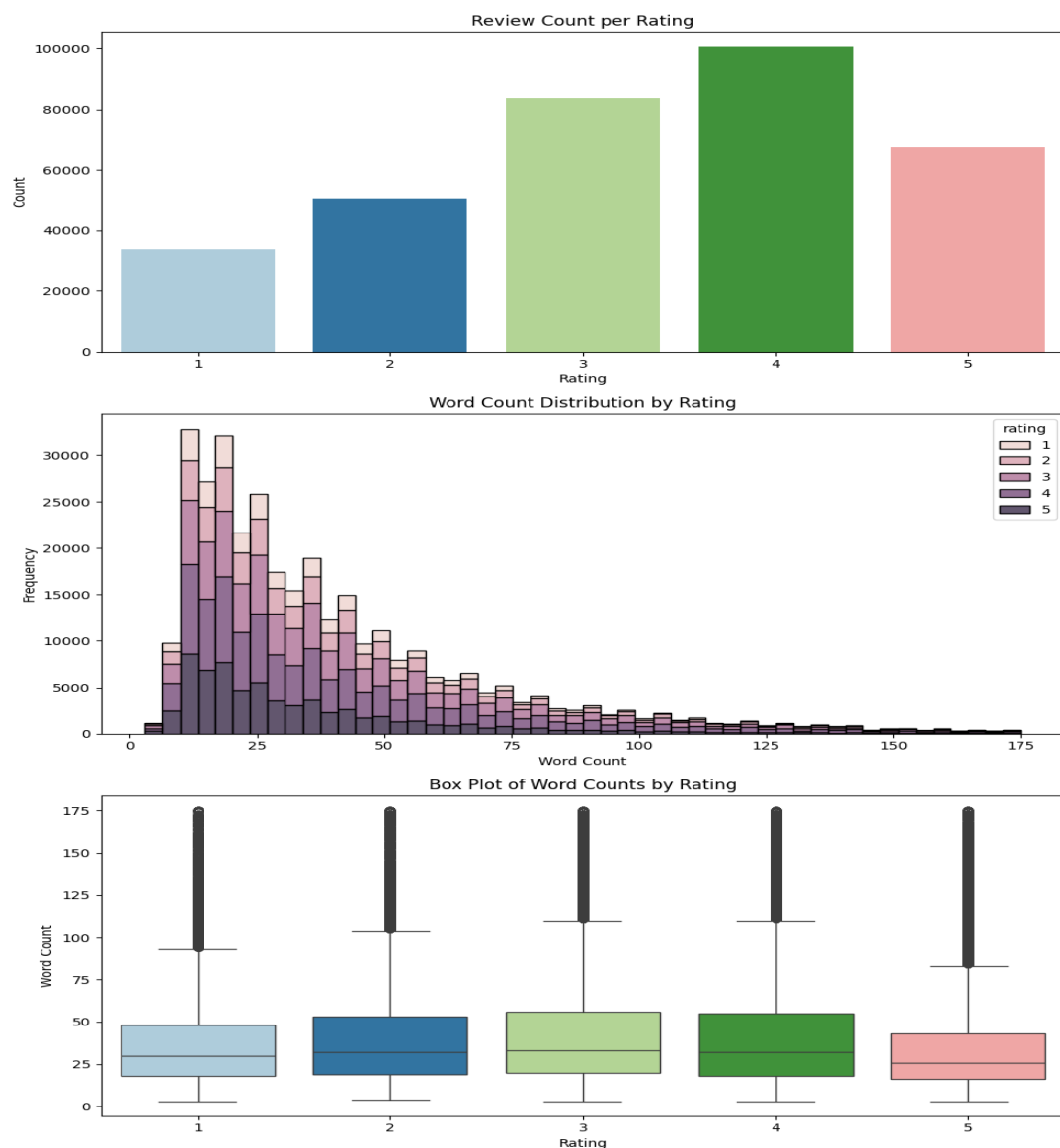
The review texts above word below 3 and above 175 is removed. The codes given below.

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
def preprocess(text)
    # lowercase
    text = text.lower()
    # remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)
    # remove HTML tags
    text = re.sub(r"<.*?>", "", text)
    # remove emojis and non-alphabetic characters
    text = re.sub(r"[^a-z\s]", "", text)
    # tokenize
    tokens = text.split()
    # remove stopwords & lemmatize
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in
stop_words]
    # rejoin
    return " ".join(tokens)
data["cleaned_review"] = data["review_text"].astype(str).apply(preprocess)
data["word_count"] = data["cleaned_review"].apply(lambda x: len(x.split()))
```


6.4 Data Visualization

6.4.1 Subplot consisting of a barplot , histogram and boxplot

```
fig, axes = plt.subplots(3, 1, figsize=(10, 15))
sns.countplot(x="rating", data=data, palette="Paired", ax=axes[0]) #bar plot
axes[0].set_xlabel("Rating")
axes[0].set_ylabel("Count")
sns.histplot(data, x="word_count", hue="rating", multiple="stack", bins=50,
ax=axes[1])
axes[1].set_title("Word Count Distribution by Rating") #histogram
axes[1].set_xlabel("Word Count")
axes[1].set_ylabel("Frequency")
sns.boxplot(x="rating", y="word_count", data=data, palette="Paired", ax=axes[2])
axes[2].set_title("Box Plot of Word Counts by Rating") #box plot
axes[2].set_xlabel("Rating")
axes[2].set_ylabel("Word Count")
plt.tight_layout()
plt.show()
```



The inferences from the plot;

Bar Plot

1. 4-star reviews are the most frequent, followed by 3-star and 5-star ratings and 1-star and 2-star reviews are comparatively fewer.
2. most users are moderately satisfied, with fewer extreme positive or negative experiences.
3. The data distribution is right-skewed, meaning the sentiment overall tends to be positive to neutral.

Histogram

1. The majority of reviews have fewer than 50 words, regardless of the rating.
2. The distribution is right-skewed, most people write short reviews, and only a few write long ones.
3. There isn't a strong difference in word count distribution across ratings (the curves overlap). It suggests that review length does not significantly depend on sentiment or rating. short and long reviews occur across all ratings.

Barplot

1. Median word count is quite similar across all ratings (roughly 25 - 35 words).
2. Outliers exist in every rating category, showing that a few users write very long reviews (above 100 words).
3. Slightly shorter median for 5-star reviews indicating happy customers may write shorter, more direct reviews, while lower ratings might involve longer explanations or complaints.
4. The variation (IQR) is consistent across all ratings, meaning review length behavior is uniform across sentiments.

6.4.2 Word Clouds for each rating.

Generated word clouds for each rating.



The brief summary of the key inferences from the word cloud image:

1. Negative and Positive Focus: Lower ratings (1 and 2) frequently feature words like "product," "flavor," and "taste," suggesting these are key issues for dissatisfaction. Higher ratings (4 and 5) emphasize positive words like "love," "good," and verbs like "use" and "make."
2. Generic vs. Evaluative: The words in the lower rating word clouds (1, 2, 3) are generally more specific to the item (product, flavor, taste, coffee, tea), indicating a direct reaction to the item's properties. In contrast, the high ratings (4, 5) frequently feature highly positive evaluative language (love, good, great, delicious).
3. Ambiguity in Middle Ratings: Rating 3 (the neutral rating) contains a mix of terms like "product," "taste," and "good," suggesting mixed feelings or non-extreme experiences.
4. Prominence of "One": The word "one" is notably large across multiple rating levels (1, 2, and 4), indicating a common use of the word in phrases like "the one," "only one," or "bought one," but its context likely shifts from negative to positive.

6.5. Sample Reviews from different ratings.

5 Sample reviews of 5 ratings are printed with the code

```
# preview of sample reviews per rating
for r in sorted(data["rating"].unique()):
    print(f"sample review of rating {r}")
    display(data[data["rating"] == r]["cleaned_review"].sample(min(5,
len(data[data["rating"] == r]))).to_list())
```

sample review of rating 1

1. 'dont know charge bundling package got order wrong twice received half order paid twice nowi hope someone else happen thats writing bad review',
2. 'disappointed find tree purchased gift someone died continuous care month',
3. 'product complete ripoff gave percent paid product complete ripoff',
4. 'avid keurig user tassimo convert thrilled finally find starbucks kcups arrived purchased box kcups pike placeroast amazon taste didnt disappoint satisfied feel rival taste cup purchased starbucks store unfortunately price also rival cup purchased starbucks storeheres math drinking caribou blend coffee pointcaribou blend average cupstarbucks cupwe understand starbucks premium coffee dont expect cheap caribou also premium coffee afford either principle suspecting starbucks corporate folk set price weve decided stay caribou
5. based household coffee consumption would pay whopping year starbucks going vote stick caribou kindle owner feel like publisher price gouging experienced set price amazon', 'love use pretzel decorate give friend family holiday recently bought case box contained least broken pretzel bag whole pretzel im glad amazon able working dilemma still extremely disappointed condition received']

sample review of rating 2

1. 'soup two specific direction cooking microwave without microwave although direction preparing rather simple dealing easy cooked microwave bowl hot rather cumbersome hold onto draining problem may need protectionthe bowl soup nutrition fact claim calorie total fat ogm saturated trans fat sodium high contains gm carbohydrate purchased lack fatyou add chicken tofu stir fried vegetable fresh chopped vegetable keep mind meal come small packet dried tofu carrot spinach topping vegetable packet consists small piece dry dehydrated appetizingthere odd flavor consistency noodle sauce tasty flavorful first impression critical im sure would try annie chuns dish crave teriyaki noodle bowl rizzo',
2. 'placed order one oz bag mm receive bag oz peanut mm although mean received oz overall product listing inaccurate',
3. 'disappointing product plenty salt much truffle flavor aroma mainly tiny percentage content made truffle piece dont know whether rival product using one doesnt add much flavor food aside saltiness',
4. 'hopeful crunchy low carb snack gave try okay salty doubt reorder',
5. 'enjoyed original fig newton new cookie pleasing tasted like sweet cracker odd chewy fig bit speckled throughout poor imitation mind']

sample review of rating 3

1. 'two traditional striped cat eat mostly dry science diet cat food nucat vitamin added also get iams canned chicken turkey beef day brownie prefers dry food eat canned food freshly opened pi like tried petite cuisine variety pack yellowfin snapper tuna sole tuna shrimp petite cuisine variety pack chinese chicken chicken pot pie course sixteen day cat got onequarter day alternated can chicken fish varietiesat first feline enthusiastic tired chicken pi would lick gravy brownie refused even look fish better received like yellowfin tuna shrimp cat vote two six varieties there however question mercury ocean fish study japan found cat ate tuna higher concentration mercury fur cat ate dry cat food study result discussed article find web using search term tuna fish posted question petite cuisine web site whether tested product mercury received answer conclusion may ok give cat fish occasional treat steady diet',
2. 'bad cheesy would hoped remember better kid',
3. 'tried year ago thought great certainly much better chocolate local grocery store recently though ive tried ritter sport decided better looking something thats better get local grocery store youre definitely treat might want give ritter try also',
4. 'must admit really excited try cooky love chocolate could better extra ooey gooey chocolate righthonestly impressed seem dry kind fake tasting arent horrible definitely good sounded think stick regular chip ahoy',
5. 'though gel functional use long run mile cant say im fond find often get stomach cramp hour following run warned possible told take water problem stay hydrated run always take gu plenty water found make difference way feel postrun run feel great trade one evil another']

sample review of rating 4

1. 'easy add water one egg simple easy make minute adding item mix doesnt provide result basic turned good',
2. 'great bold coffee avid starbucks drinker prefer darker roast one fit bill mei also like timothy midnight magictimothys rainforest espresso coffee people bold black tiger gloria jean black gold',
3. 'mix wheat germ flaxseed milk every morning love beneficial every part body wouldnt think going without highly recommended wheat germ',
4. 'knew taste pleasure roller coaster disappointed brewingmy tastebud began waterwhat liked flavored coffee didnt get back taste often get coffee even coffee cooled still delicious sometimes like chill coffee',
5. 'unlike bar crunchy theyre loaded nut seed husband love wish organic natural mean healthy know']

sample review of rating 5

1.'spent time searching alcohol free rum flavor make daiquiri sorbet lorann produced best flavor thus far taste outstanding blow away competitor tried baking yet attempted imagine would similar result price quite reasonable oz bottle although much multiple bottle ordered save shipping natural artificial ingredient would prefer natural thats price im willing pay flavor predict remain alcohol free rum flavor choice',

2.'started using purina second nature dog litter one day brought puppy home breeder week old within day trained use litter box thought product bit expensive litter soon found worth used various cheaper brand period month order save money found werent absorbent untrackable important odor none tried actually stunk house thrown away immediately refuse use anything second nature im satisfied result',

3. 'organic instant coffee rich enough taste good camping put emergency stash supplies intend put jar gift basket got first place k cup starting environmental problem wanted quick morning cup joe without causing guilt instance',

4.'accidentally bought cheese mistake instead original version pleasantly surprised robust aromatic delightful',

5. 'happy ordered swanson himalayan pink salt wonderful product many health benefit felt quality product outstanding impressed endorsed dr barbara hendel thank swanson bringing wonderful nutrient usa pakistan brand contains pure himalayan salt fake rock salt like company price reasonable']

6.6. Train Test Split

The dataset is split into training (80%) and testing (20%) sets using the `train_test_split()` function. The `stratify=y` parameter ensures that the proportion of each rating category remains the same in both the training and test sets.

```
x = data["cleaned_review"] # training
y = data["rating"]         # testing

x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.2, stratify=y, random_state=42)

print("Train set:\n", y_train.value_counts().to_dict())
print("Test set:\n", y_test.value_counts().to_dict())
```

6.7. TF-IDF Vectorization

TF-IDF is a statistical measure that evaluates how important a word is in a document relative to a collection of documents (corpus). It combines two components:

1. Term Frequency (TF): Measures how often a word appears in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. Inverse Document Frequency (IDF): Measures how unique or rare a word is across all documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } t}$$

3. TF-IDF Score:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

Words that occur frequently in a document but rarely across all documents get higher TF-IDF scores, highlighting their importance for classification. Common words like "the" or "is" get low scores because they appear in almost all documents.

This TF-IDF representation allows the model to focus on the most relevant words for predicting review ratings

```
#converts text data into numerical features using TF-IDF(Term Frequency Inverse Document Frequency)

vectorizer = TfidfVectorizer(max_features=5000) #limits the number of features/unique words to the top 5000 most imp words in dataset.
X_train_tfidf = vectorizer.fit_transform(x_train)
X_test_tfidf = vectorizer.transform(x_test)

print("TF-IDF Train shape:", X_train_tfidf.shape)
print("TF-IDF Test shape:", X_test_tfidf.shape)
```

And the output was;

TF-IDF Train shape: (269064, 5000) and TF-IDF Test shape: (67267, 5000)

Model Training and Evaluation.

Model Selection Summary

In this project, five supervised machine learning models were developed and evaluated for sentiment classification: Logistic Regression, Linear Support Vector Machine (SVM) and Naive Bayes

These models were selected because they represent a balanced mix of linear, probabilistic, and ensemble learning approaches, each offering unique strengths for text-based sentiment analysis.

1. Logistic Regression:

Logistic Regression is a strong and interpretable baseline classifier for text classification tasks. It performs well with high-dimensional sparse data generated from TF-IDF or count vectorization, and provides probabilistic outputs useful for understanding class confidence levels.

2. Linear SVM:

Support Vector Machines are known for their ability to handle high-dimensional feature spaces efficiently. A Linear SVM finds an optimal separating hyperplane between sentiment classes, making it especially suitable for text data where the number of features (words) is much greater than the number of samples.

3. Naive Bayes:

Naive Bayes classifiers, particularly MultinomialNB, are commonly used in document classification and sentiment analysis due to their simplicity, speed, and strong performance on sparse text features. It assumes independence between features, which, despite being a simplification, often works well in practice for word-based models.

Model Training and Optimization Process

To improve performance **optimized TF-IDF feature extraction** approach was implemented. The following modifications were made to the TF-IDF vectorizer:

```
vectorizer = TfidfVectorizer(  
    max_features=15000, # expand vocabulary      (before 5000)  
    ngram_range=(1,2), # include bigrams  
    min_df=3,          # ignore very rare words  
    max_df=0.9         # ignore overly common words  
)  
# Increasing max_features and adding bigrams can capture more contextual  
information.
```


This optimization helped the models capture richer text patterns, such as combinations of words (bigrams), and reduced noise from very rare or overly common words.

Hyperparameter Tuning

After improving the feature extraction, **hyperparameter tuning** was performed individually for each model using **GridSearchCV** to find the best parameter combinations that maximize model performance. The tuning process helped achieve better accuracy and balance across different sentiment classes. Cross-validation (CV) was also included to evaluate performance and ensure the model's results are stable across different subsets of the data.

```
# Hyperparameter tuning using GridSearchCV
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(model, param_grid=params, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_tfidf, y_train)
print("Best Params:", grid.best_params_)
```

Model Implementation and Results

1. Logistic Regression (LR):

Why this model:

- Logistic Regression is a strong baseline model for text classification problems, including sentiment and review rating prediction.
- It works well with high-dimensional and sparse data like TF-IDF vectors, where most features (words) are zero for each review.
- LR provides interpretable coefficients, allowing insight into which words most influence each rating.
- It's computationally efficient and often performs comparably to more complex models when properly tuned.

```
# Hyperparameter tuning
lr_params = {
    'C': [0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs', 'liblinear'],
    'class_weight': [None, 'balanced']
}

lr_grid = GridSearchCV(
```

```

        LogisticRegression(max_iter=1000),
        lr_params,
        scoring='accuracy',
        cv=3,
        n_jobs=-1,
        verbose=1
    )
lr_grid.fit(X_train_tfidf, y_train)

# Evaluate
lr_best = lr_grid.best_estimator_
y_pred_lr = lr_best.predict(X_test_tfidf)

print("\n Logistic Regression Best Params:", lr_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_lr), 4))
print(classification_report(y_test, y_pred_lr, digits=4))

cm = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
and the output was;

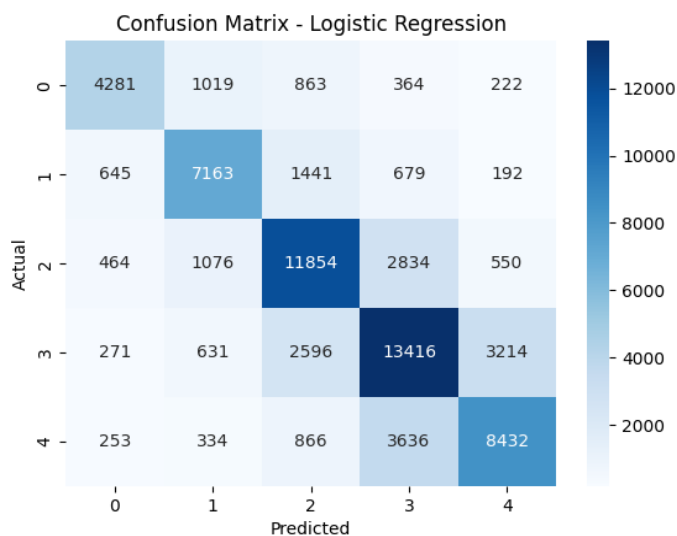
```

```

Logistic Regression Best Params:
{'C': 10, 'class_weight': None, 'penalty': 'l2', 'solver': 'lbfgs'}
Accuracy: 0.6709

```

	precision	recall	f1-score	support
1	0.7239	0.6343	0.6761	6749
2	0.7007	0.7078	0.7042	10120
3	0.6728	0.7065	0.6892	16778
4	0.6410	0.6665	0.6535	20128
5	0.6687	0.6236	0.6454	13521
accuracy			0.6709	67296
macro avg	0.6814	0.6678	0.6737	67296
weighted avg	0.6718	0.6709	0.6707	67296



Logistic Regression – Inference

Best Parameters: {'C': 10, 'class_weight': None, 'penalty': 'l2', 'solver': 'lbfgs'}

Accuracy: 67.09%, indicates a fair ability to classify the review ratings based on text features (TF-IDF vectors).

The macro and weighted averages of precision, recall, and F1-score are consistent (~0.67), showing that the model performs fairly evenly across all rating classes.

Confusion Matrix Insights

The matrix shows strong diagonal dominance, indicating decent classification accuracy overall.

Misclassifications are mostly between neighboring ratings (e.g., 3-4, 2-3), which is common in ordinal sentiment data where boundaries between classes are subtle.

The model rarely confuses extreme classes (1 and 5), suggesting it captures sentiment polarity reasonably well

Overall,

Logistic Regression provides a balanced baseline model with steady results across classes.

However, the model might struggle to capture nuanced language features or contextual sentiment shifts since TF-IDF + Logistic Regression are linear and do not capture deeper semantics.

2. Linear SVM

Why this model;

Linear SVMs are powerful for text classification, especially when feature vectors are high-dimensional.

They aim to find the optimal separating hyperplane, maximizing the margin between classes, which usually leads to better generalization.

SVMs are robust to overfitting, handle non-linear relationships through kernel tricks (though linear kernel is ideal for text), and often outperform other linear classifiers on textual data.

This makes SVMs an ideal candidate for identifying subtle sentiment differences in reviews.

```
# Hyperparameter tuning
svm_params = {
    'C': [0.1, 1, 10],
    'class_weight': [None, 'balanced'],
    'loss': ['hinge', 'squared_hinge']
}

svm_grid = GridSearchCV(
    LinearSVC(max_iter=2000),
    svm_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
svm_grid.fit(X_train_tfidf, y_train)

# Evaluate
svm_best = svm_grid.best_estimator_
y_pred_svm = svm_best.predict(X_test_tfidf)

print("\n Linear SVM Best Params:", svm_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_svm), 4))
print(classification_report(y_test, y_pred_svm, digits=4))

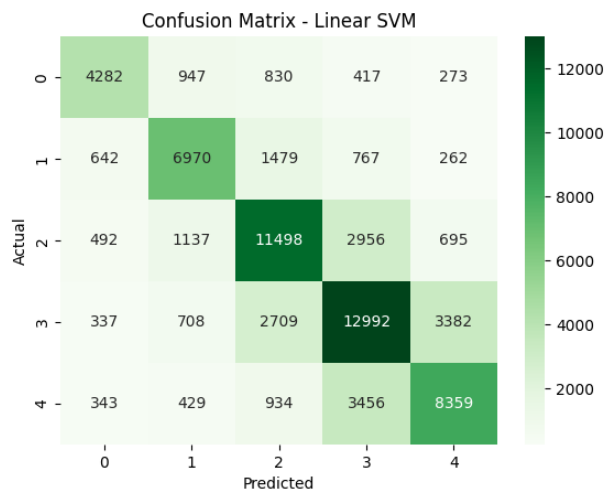
cm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix - Linear SVM")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
plt.show()
```

and the output was;

Linear SVM Best Params: {'C': 10, 'class_weight': None, 'loss': 'squared_hinge'}

Accuracy: 0.6553

		precision	recall	f1-score	support
	1	0.7024	0.6345	0.6667	6749
	2	0.6839	0.6887	0.6863	10120
	3	0.6589	0.6853	0.6718	16778
	4	0.6310	0.6455	0.6382	20128
	5	0.6444	0.6182	0.6311	13521
	accuracy			0.6553	67296
	macro avg	0.6642	0.6544	0.6588	67296
	weighted avg	0.6558	0.6553	0.6552	67296



Linear SVM – Inference

- Best Parameters: {'C': 10, 'class_weight': None, 'loss': 'squared_hinge'}
- **Accuracy: 65.53%**, which is slightly lower than Logistic Regression (67.1%).
- The Linear SVM model achieved an overall accuracy of 65.5%, which is slightly lower than Logistic Regression (67.1%).
- The macro and weighted averages of precision, recall, and F1-score are all around 0.65, indicating balanced but modest performance.

Confusion Matrix Insights

- The confusion matrix shows a similar pattern to Logistic Regression, with the major errors between adjacent ratings (3-4, 2-3).
- The model maintains reasonable diagonal strength, confirming it correctly identifies most classes.
- Misclassifications are less random but more concentrated around mid-scale ratings, suggesting SVM has difficulty distinguishing between moderate ratings.

Overall,

- The SVM performs comparably to Logistic Regression but shows slightly lower accuracy and F1-scores across most classes.
- Despite being a strong linear classifier, Linear SVM doesn't provide a significant advantage over Logistic Regression when both use TF-IDF features — this suggests that the dataset may not have strong linear separability.
- Both models face the same challenge, confusion between adjacent ratings, indicating that deeper or nonlinear models might better capture nuanced sentiment patterns.

3. NAIVE BAYES

Why this model;

- Naive Bayes is a probabilistic model that assumes feature independence a simplifying assumption that surprisingly works well in text analysis.
- It's extremely fast and scalable, making it efficient for large review datasets.
- The Multinomial variant is well-suited for TF IDF or count-based features, modeling word occurrence probabilities across classes.
- It often serves as a benchmark for text classification due to its simplicity and decent performance.

```
nb_params = {
    'alpha': [0.1, 0.5, 1.0, 2.0],      # smoothing parameter
    'fit_prior': [True, False]
}

nb_grid = GridSearchCV(
    MultinomialNB(),
    nb_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
nb_grid.fit(X_train_tfidf, y_train)

# Evaluate
nb_best = nb_grid.best_estimator_
y_pred_nb = nb_best.predict(X_test_tfidf)

print("\n Naive Bayes Best Params:", nb_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_nb), 4))
print(classification_report(y_test, y_pred_nb, digits=4))

# Confusion Matrix
```

```

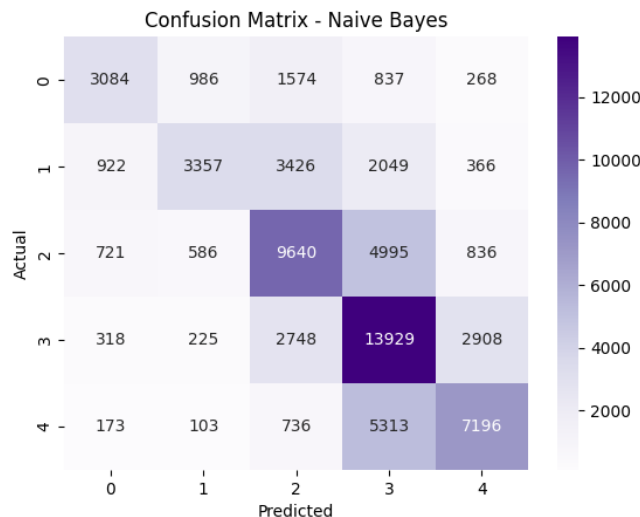
cm = confusion_matrix(y_test, y_pred_nb)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples')
plt.title("Confusion Matrix - Naive Bayes")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

and the output was;

Naive Bayes Best Params: {'alpha': 0.1, 'fit_prior': True}
Accuracy: 0.5529

		precision	recall	f1-score	support
	1	0.5910	0.4570	0.5154	6749
	2	0.6386	0.3317	0.4366	10120
	3	0.5319	0.5746	0.5524	16778
	4	0.5135	0.6920	0.5896	20128
	5	0.6217	0.5322	0.5735	13521
	accuracy			0.5529	67296
	macro avg	0.5794	0.5175	0.5335	67296
	weighted avg	0.5664	0.5529	0.5466	67296



Naive Bayes - Inferences

- Best Parameters: { alpha = 0.1, fit_prior = True}
- Accuracy: 55.29%, indicates the model captures some signal but struggles to fully distinguish between classes
- Rating 1 & 2: Lower recall (0.45 & 0.33) means the model struggles to identify very low ratings.
- Rating 3: Balanced precision and recall, performs best here.
- Rating 4: High recall (0.69) tends to overpredict this class.
- Rating 5: Moderate precision (0.62) but recall only 0.53, misclassifies some positive reviews.

Confusion Matrix Insights

- Most reviews rated 3 or 4 are correctly identified (the matrix's darkest regions.)
- Model confuses 3 and 4 heavily, which makes sense since their wording patterns are often similar ("good" vs "very good").
- Lower ratings (1 & 2) get scattered across other classes, especially misclassified as 3.
- Class 5 also leaks into class 4.

Overall,

- Naive Bayes performs reasonably for a baseline text model but shows class imbalance or semantic overlap issues.
- Among the three models, **Logistic Regression** demonstrated the highest accuracy; therefore, it was chosen as the final model for prediction.